

Week 9

1. UNIX Signals (I)

Dr. Andreas S. Maniatis
Assistant Professor

School of Computer Science
COMP.2560 – System Programming [2025F]

Monday, October 27th, 2025

DEFYXHQGRUOSGRP RN	3236343972474116
QFJWQJXVRI DKUEVP	893833235594578
KGWBGQFQHP RSUP RU	0310019701552750
XURNYUWZ DCSYUUM I	7875465400196596
WHMFSP UIQYSUJYRSHRG	5019353606937419



University of Windsor

Content

- 1 Introduction
- 2 Signal concepts
- 3 Handling signals

Part I

Introduction to Signals



Signals

Signals are one way of performing **inter-process communication** (in addition to pipes and sockets).

Why signals?

Programs must sometimes deal with unexpected or unpredictable events:

- A termination request from the user (CTRL-C).

- A suspend request from the user (CTRL-Z).

- The death of a child process.

- A floating-point error.

- An alarm clock.

- A power failure.

- ...

These events are sometimes called **interrupts** because they must interrupt the program to be processed.

Who can send a signal?

When such an event occurs, **the kernel** sends the appropriate process an event-specific **Signal**.

A process can send a signal to another process (as long as it has permission). For example, a parent process can send a "kill" signal to its child.

The defined signals

Every signal has a name that begins with `SIG`.

Signal names are all defined by positive integer constants in `</usr/include/bits/signum-generic.h>`.

For example: `SIGABRT`, `SIGINT`, `SIGALRM`,...

Part II

Signal Concepts



Handling a signal

A programmer may arrange for a particular signal to be **ignored** or **processed** by means of a special piece of code called **signal-handler**.

What can a programmer do?

Note that a programmer may choose for a particular signal to:

- trigger a **user-supplied** `signal-handler` or,
- trigger the **default kernel-supplied** handler or,
- be **ignored**. This works for most signals **except** the two signals: **SIGKILL** and **SIGSTOP** that cannot be ignored.

The default kernel-supplied handler

The default handler usually performs one of the followings:

- **terminates** the process and generates a core file (`dump`).
- **terminates** the process without generating a core file (`quit`).
- **ignores** signal (`ignore`).
- **suspends** process (`suspend`)
- **resumes** process: if the process was stopped, the default action is to continue the process, otherwise, the signal is ignored.

A list of signals... (always use the name, not the value in your program)

Name	Value	Action	Description
SIGHUP	1	quit	hang-up
SIGINT	2	quit	interrupt
SIGQUIT	3	dump	quit
SIGILL	4	dump	invalid instruction
SIGTRAP	5	dump	trace trap(used by debuggers)
SIGABRT	6	dump	abort
SIGEMT	7	dump	emulator trap instruction
SIGFPE	8	dump	arithmetic exception
SIGKILL	9	quit	kill (fatal)
SIGBUS	10	dump	bus error (bad format address)
SIGSEGV	11	dump	segmentation fault
SIGSYS	12	dump	bad argument to system call
SIGPIPE	13	quit	pipe/socket error
SIGALARM	14	quit	alarm clock
SIGTERM	15	quit	termination signal

A list of signals (cont'd) CAUTION: the values are platform dependent

Name	Value	Action	Description
SIGUSR1	16	quit	user signal 1
SIGUSR2	17	quit	user signal 2
SIGCHLD	18	ignore	child status changed
SIGPWR	19	ignore	power fail
SIGWINCH	20	ignore	window size change
SIGURG	21	ignore	urgent socket condition
SIGPOLL	22	exit	pollable event
SIGSTOP	23	suspend	stops a process
SIGSTP	24	suspend	interactive stop signal(CTR-Z)
SIGCONT	25	ignore	Continue executing if stopped
SIGTTIN	26	quit	Background process attempt read
SIGTTOU	27	quit	Background process attempt write
SIGVTALARM	28	quit	timer expired
SIGPROF	29	quit	profiling timer expired
SIGXCPU	30	dump	CPU time limit exceeded
SIGXFSZ	31	dump	file size limit exceeded

CTRL-C and CTRL-Z

The easiest way to send a signal to a foreground process is to press CTRL-C or CTRL-Z.

By pressing CTRL-C, a signal **SIGINT** is sent to all processes in the current foreground job. By default, **SIGINT terminates** a process.

By pressing CTRL-Z, a signal **SIGSTP** is sent to all processes in the current foreground job. By default, **SIGSTP suspends** a process.

SIGALRM

SIGALRM is an alarm clock signal.

This signal can be sent using `alarm()`.

The default handler of **SIGALRM** displays "Alarm clock" and **terminates** the process receiving the signal.

`alarm()`

Synopsis: **`unsigned int alarm(unsigned int n)`**

Asks the kernel to send a **SIGALRM** to the calling process after `n` seconds.

- If a previous alarm has been scheduled, it will be rescheduled: alarm requests are not stacked.
- If `n=0`, any pending alarm will be cancelled.
- If there is a previous alarm request with time remaining, `alarm()` returns a non-zero value that is the number of seconds until the previous request would have generated a **SIGALRM** signal. Otherwise, `alarm()` returns 0.

man alarm

Example (alarm.c)

Example

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[]){
    alarm(4);
    while(1){
        printf("Going to sleep\n");
        sleep(1);
    }
    printf("Exiting\n");//will not be executed
    exit(0);
}
```

```
Going to sleep
Going to sleep
Going to sleep
Going to sleep
Alarm clock
```

alarm() and fork() (alarm2.c)

A call to `fork()` **clears** pending alarms in the child process.

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(){
    alarm(4);
    int pid=fork();
    while(1){
        if(pid==0)printf("I am the child\n");
        else printf("I am the parent\n");
        sleep(1);
    }
}
```

Output of the previous example

```
I am the parent
I am the child
I am the child
I am the parent
I am the child
I am the parent
I am the child
I am the parent
Alarm clock
I am the child
I am the child
I am the child
...
```

Note that the call to `alarm()` had no effect on the child process.

alarm() and exec() (alarm3.c, donothing.c)

A new process image created by one of the **exec system calls** inherits the time left to an alarm signal in the old process's image.

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(){
    alarm(4);
    if(fork()==0){
        alarm(5);
        printf("I am the child\n");
        sleep(2);
        execl("./do_nothing","do_nothing",NULL);
    }
    else while(1){
        printf("I am the parent\n");
        sleep(1);
    }
}
```


Output of the previous example

```
I am the parent
I am the child
I am the parent
Do nothing program
I am the parent  Do
nothing program  I
am the parent
Alarm clock
Do nothing program
```

The `donothing.c` program terminates because of the `alarm(5)` call. It terminates after the time left when `exec` is called (roughly about $5 - 2 = 3$ seconds).

Part III

Handling Signals



Handling signals

A process can specify what to do when a particular signal is received.

In particular, the `signal()` system call may be used to override the default kernel-supplied action (or handler).

`signal()`

Synopsis:

`void (*signal (int signo, void (*func) (int))) (int)`
(1410 COMP?)

This system call has two arguments:

`int signo`: the signal number to be reprogrammed, a constant from the previous table.

`void (*func) (int)`: the address of a user-defined function, or
`SIGIGN` to ignore a signal (except `SIGKILL` and `SIGSTOP`), or
`SIGDFL` to use the default handler.

About the prototype of `signal()`

```
void (*signal(...))1(int)2;
```

The pair ()₁ is there because `signal()` returns a function, not a value.

(int)₂ is there because `signal()` returns a function that takes one parameter of type `int`.

Because of the `*`, `signal()` returns the address (pointer) of a function.

What does the `signal()` system call exactly return?

If successful, `signal()` returns the previous `func()` value associated with `signo` (the previous handler) or, `SIG_ERR` (i.e. -1) otherwise.

Example: ignoring CTRL-C and CTRL-Z(ctr1_cz.c)

```
int main(int argc, char *argv[]){
    int i;
    void (*oldHandler1)(); //to save default handlers
    void (*oldHandler2)(); //for CTRL-C and CTRL-Z

    oldHandler1=signal(2, SIG_IGN); //ignore CTRL-C
    oldHandler2=signal(24, SIG_IGN); // ignore CTRL-Z
    for(i=1; i<=10; i++){
        printf("I am not sensitive to CTRL-C/CTRL-Z\n");
        sleep(1);
    }
    signal(2, oldHandler1); // restore default
    signal(24, oldHandler2); // restore default
    for(i=1; i<=10; i++){
        printf("I am sensitive to CTRL-C/CTRL-Z\n");
        sleep(1);
    }
}
```

Example 1: replacing a default handler by our own one-time handler (alarm_handler.c)

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void alarm_handler(int);

void alarm_handler(int dummy){
    printf("I got an alarm, I took care of signal %d\n", dummy);
    alarm(3);
}

int main(int argc, char *argv[]){

    signal(14, alarm_handler); //install handler
    alarm(3);

    while(1){
        printf("I am working\n");
        sleep(1);
    }
}
```

Example 1: replacing a default handler by our own one-time handler (alarm_handler.c)

Output of the previous example

```
I am working
I am working
I am working
I got an alarm, I took care of signal 14
I am working
I am working
I am working
... .
```

Example 2: replacing a default handler by our own

alarm_handler2.c

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void alarm_handler(int);

void alarm_handler(int dummy){
    printf("I got an alarm, I took care of signal %d\n", dummy);
    signal(14, alarm_handler); //install handler
    alarm(3);
}

int main(int argc, char *argv[]){

    signal(14, alarm_handler); //install handler
    alarm(3);                  // for first time

    while(1){
        printf("I am working\n");
        sleep(1);
    }
}
```


Example 2: replacing a default handler by our own

Output of the previous example

```
I am working
I am working
I am working
I got an alarm, I took care of signal 14
I am working
I am working
I am working
I got an alarm, I took care of signal 14
I am working
I am working
I am working
I got an alarm, I took care of signal 14
I am working
I am working
...
```

signal() and fork() (signal_fork.c)

A child process inherits the signal setting from its parent during a fork().

```
int main(int argc, char *argv[]){
    int i;
    void (*oldHandler1)(); //to save default handlers
    void (*oldHandler2)(); //for CTRL-C and CTRL-Z

    oldHandler1=signal(2, SIG_IGN); //ignore CTRL-C
    oldHandler2=signal(24, SIG_IGN); // ignore CTRL-Z

    if(fork() !=0)
        for(i=1; i<=10; i++){
            printf("Parent: I am not sensitive to CTRL-C/CTRL-Z\n");
            sleep(1);
        }
    else while(1){
        printf("Child: I am not sensitive to CTRL-C/CTRL-Z?\n");
        sleep(1);
    }
}
```

Both the child and parent are insensitive to CTRL-C/CTRL-Z.

signal() and exec()

When a process performs an `exec()`,
previously ignored signals remain ignored,
installed handlers are set back to the default handler.

See assignment /lab?

When `exec(...)` ...
Ignored ignored
Installed restored

Stacked vs. non stacked signals

Except `SIGCHLD`, signals are not stacked. Therefore, if a number of signals are sent to a process only one of the signals is actually processed.

Problem to think about

Modify the Mini Shell Program we have studied in the previous lecture so the terminating child processes running in the background do not become zombies.

Hint: study the `SIGCHLD` signal.

SIGCHLD

Whenever a process terminates or stops, the SIGCHLD signal is sent to the parent. By default, this signal is ignored, so the parent must catch this signal if it wants to be notified whenever a child's status changes. The normal action in the signal-catching function is to call one of the `wait` functions to fetch the child's process ID and termination status.

Earlier releases of System V had a similar signal named SIGCLD (without the H). The semantics of this signal were different from those of other signals, and as far back as SVR2, the manual page strongly discouraged its use in new programs. (Strangely enough, this warning disappeared in the SVR3 and SVR4 versions of the manual page.) Applications should use the standard SIGCHLD signal, but be aware that many systems define SIGCLD to be the same as SIGCHLD for backward compatibility. If you maintain software that uses SIGCLD, you need to check your system's manual page to see which semantics it follows. We discuss these two signals in Section 10.7.

Part IV

Assignments #3, #4: Overview



Assignment #3: Overview

- It's all about Coding!
- Read your examples
- Access your textbook
- Work methodology:
 - Start as early as possible
 - Read carefully!
 - Read again!
 - Reach out for clarifications early
 - Make your example easily readable
- Submit way before the deadline
- Avoid using the grace period



Assignment #3: Overview

- It's all about coding
- Questions on Process Control:
 - Bit-manipulation
 - Write program
 - Re-write program
- Make sure it compiles
- Make sure the logic is correct
- Submission:
 - Code
 - Video, explaining what you did



Assignment #4: Overview

- It's all about coding
- Questions on Signals:
 - Bash Tutorial Certificate from DataCamp
 - Write program
 - Re-write program
- Make sure it compiles
- Make sure the logic is correct
- Submission:
 - Code
 - Video, explaining what you did



Part V
Attendance Quiz (Not graded)



Quiz



Attendance Quiz

701968234901768023405879312016
670980321768767



774862586034187069210783468019
701968234901768023405879312016
670980321768767

WPOYFXI LFJXI G I E
H I MPHLOM ZFOLI PG
BXNYSXWHYGI JLG I L
OLI EPC OOUJPL QDZC
NYDWIGL OUPJL QDZC

Thank you!
Questions?

DEFYXHQGRUOSGRP RN	3236343972474116
QJRMVQJXVRI DKUEVP	893833239594578
KGWBGQFHP RSUP RU	0310019701552750
XURNYUXZ DCSYUUM I	7875465400196596
WHMFSP UIQYSUQYRSHRG	5019353606937419

00110011100111
11111010000100
11000000001111
00111100111100
00101111111100
00000111011011
11001100111100

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor